

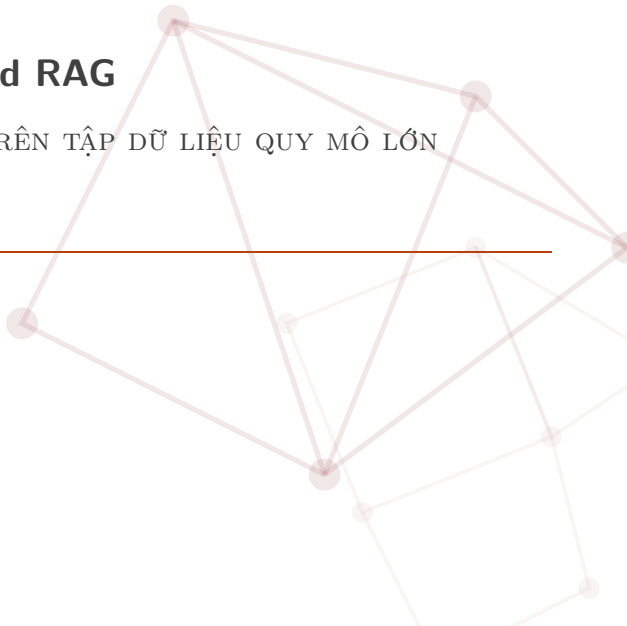
T-RAG: Temporal & Targeted RAG

KHẮC PHỤC HẠN CHẾ CỦA RAG TRÊN TẬP DỮ LIỆU QUY MÔ LỚN
(ENTERPRISE RAG-BENCH)

Người trình bày: Bảo & Bằng

Hệ thống RAG Benchmark & Research

Ngày 23 tháng 6 năm 2026



Bối cảnh & Thách thức

Kiến trúc Đề xuất: T-RAG Pipeline

Thiết lập Môi trường & Đo lường (Benchmark Setup)

Bối cảnh & Thách thức



Đặc tả tập dữ liệu EnterpriseRAG-Bench

Tính chất của dữ liệu thực tế

Khác với Wikipedia, tập dữ liệu nội bộ bao gồm nhiều định dạng đa dạng, cấu trúc không đồng nhất và có tần suất cập nhật cao.

Slack

(285k docs)

Hội thoại ngắn

Gmail

(121k docs)

Luồng email (Thread)

Linear/Jira

(41k docs)

Tiến độ dự án

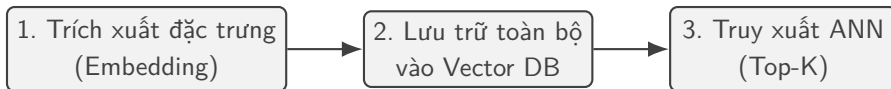
Confluence

(5k docs)

Văn bản cấu trúc dài

Quy mô tập dữ liệu: Hơn **500.000 văn bản** từ 9 nguồn dữ liệu khác nhau.

Phân tích phương pháp RAG Tiêu chuẩn (Standard RAG)



Các nguyên nhân dẫn đến suy giảm hiệu suất:

- ❌ **Mật độ Vector cao:** Với 500.000 tài liệu, khoảng cách Cosine giữa các vector trở nên kém phân biệt. Nhiều tài liệu chung chủ đề dễ bị truy xuất nhầm.
- ❌ **Thiếu nhận thức thời gian (Temporal Blindness):** Tương đồng ngữ nghĩa không phản ánh tính mới của thông tin (ví dụ: Quyết định năm 2022 so với 2024).
- ❌ **Bất đồng từ vựng (Vocabulary Mismatch):** Truy vấn của người dùng thường mang tính ngữ cảnh (contextual), trong khi văn bản gốc chứa mã định danh hoặc tên kỹ thuật.

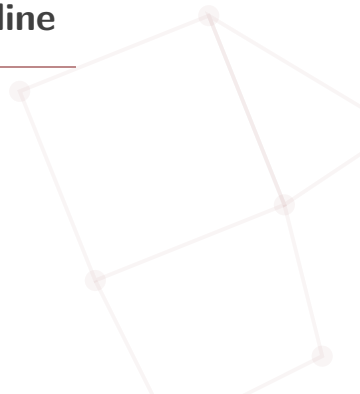
Giải pháp đề xuất: Mô hình T-RAG

✓ Phương pháp tiếp cận

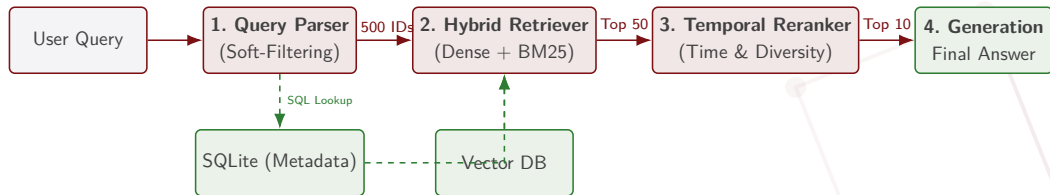
T-RAG đề xuất kiến trúc xử lý tuần tự, tích hợp tiền xử lý có định hướng (**Targeted**) và xếp hạng theo trục thời gian (**Temporal**).

- **Tiền lọc Metadata (Pre-filtering):** Sử dụng cơ sở dữ liệu quan hệ (SQL) để thu hẹp không gian tìm kiếm, loại bỏ sớm các tài liệu không liên quan trước khi tính toán độ tương đồng.
- **Phân bổ trọng số thời gian (Time Decay):** Áp dụng hàm suy giảm để giảm trọng số các tài liệu cũ, đảm bảo tính cập nhật của thông tin truy xuất.
- **Truy xuất lai (Hybrid Retrieval):** Kết hợp kỹ thuật Sparse (BM25) và Dense (Embedding) nhằm tối ưu khả năng đối khớp từ vựng và ngữ nghĩa.

Kiến trúc Đề xuất: T-RAG Pipeline



Cấu trúc Tổng quan T-RAG



Phân hệ 1: Phân tách Lưu trữ (Storage Isolation)

Relational DB (SQLite)

- **Lưu trữ Metadata:** Ngày tạo, Tác giả, Trạng thái tài liệu.
- **Cập nhật hiệu quả:** Cho phép cập nhật trạng thái văn bản nhanh chóng bằng truy vấn SQL mà không cần tính toán lại Vector.
- **Chỉ mục văn bản:** Tích hợp FTS5 hỗ trợ thuật toán BM25.

Vector DB (LanceDB)

- **Lưu trữ Embeddings:** Chuyên biệt cho biểu diễn vector và mã định danh (doc_id).
- **Tính ổn định:** Giảm thiểu tần suất ghi (write operations); chỉ cập nhật khi nội dung cốt lõi thay đổi.
- Giảm thiểu đáng kể chi phí tính toán trong quá trình Indexing.

Phân hệ 2: Truy vấn & Cơ chế Nới lỏng (Soft-Filtering)

⌵ Logic Lọc Nới lỏng (Soft-Filtering)

Việc áp dụng các điều kiện cứng (Hard-filter) thường trả về tập rỗng do lỗi trích xuất thực thể. T-RAG áp dụng cơ chế tự động nới lỏng giới hạn nhằm duy trì một tập ứng viên (Candidate List) ổn định.

Minh họa quá trình Fallback:

- ⌵ Bước 1: Khớp toàn bộ điều kiện (Date='2025' AND Author='A' AND Type='Meeting')
→ Lượng kết quả thấp → Kích hoạt Fallback (Bỏ qua Date).
- ⌵ Bước 2: Khớp một phần (Author='A' AND Type='Meeting')
→ Lượng kết quả thấp → Kích hoạt Fallback (Bỏ qua Author).
- ⌵ Bước 3: Chỉ giữ thuộc tính cốt lõi (Type='Meeting')
→ **Đạt ngưỡng ứng viên (Ví dụ: 500 documents) → Chuyển tiếp tới Hybrid Retriever.**

Phân hệ 3: Truy xuất Lai (Hybrid Retriever)

🔍 Tích hợp Dense & Sparse

Hoạt động truy xuất chỉ được thực hiện trên tập ứng viên giới hạn (Candidate List) thay vì toàn bộ cơ sở dữ liệu.

- **Dense Search (Semantic):** Đánh giá độ tương đồng Cosine, cho phép ánh xạ các khái niệm tương đương dù không trùng khớp từ vựng.
- **Sparse Search (Lexical - BM25):** Cần thiết để nhận diện chính xác các mã định danh, ID đặc thù (như mã lỗi, số ticket) mà Dense Search thường bỏ sót.
- **Hợp nhất bằng RRF (Reciprocal Rank Fusion):** Kết hợp thứ hạng từ cả hai phương pháp để trích xuất tập tài liệu liên quan nhất.

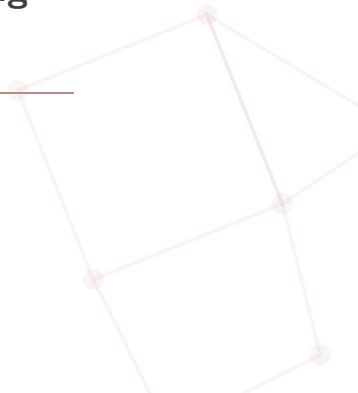
Phân hệ 4: Mô hình Reranker (Temporal & Diversity)

📅 Hàm phạt thời gian (Time Decay)

$$\text{Final_Score} = \text{Relevance} \times e^{-\lambda \Delta t} \times \text{Diversity_Penalty} \quad (1)$$

- **Relevance** ($0 \rightarrow 1$): Điểm đánh giá mức độ liên quan ngữ nghĩa từ mô hình Cross-Encoder (ví dụ: BGE-Reranker).
- **Time Penalty** ($e^{-\lambda \Delta t}$): Với Δt là khoảng lùi thời gian. Hệ số này làm suy giảm theo hàm mũ điểm số của các tài liệu cũ, ưu tiên thứ hạng cho thông tin hiện hành.
- **Diversity Penalty (MMR)**: Áp dụng thuật toán Maximal Marginal Relevance nhằm phạt các tài liệu có nội dung trùng lặp, đảm bảo tính đa dạng trong tập kết quả cuối.

Thiết lập Môi trường & Đo lường (Benchmark Setup)



Mô hình Triển khai trên GPU H100 (1)

- 🖥️ **Phần cứng:** 1 Node GPU H100 (40GB - 80GB VRAM).
- 🔗 **Nền tảng (Native Python + Asyncio):** Xây dựng luồng thực thi bất đồng bộ nhằm tối ưu hóa độ trễ (Latency) và tính minh bạch của quy trình.
- 🏆 **Mục tiêu Đánh giá:** Tối đa hóa thông lượng (Throughput) trên tập dữ liệu EnterpriseRAG-Bench để phục vụ công bố kết quả thực nghiệm.

⚙️ Phân bổ Tài nguyên VRAM

- **LLM Engine (vLLM):** Vận hành mô hình Llama-3-8B / Qwen2.5-7B, xử lý tác vụ *Query Parsing* và *Generation*. (Yêu cầu ~ 16GB). Sử dụng PagedAttention để tăng Batch Size.
- **Cross-Encoder:** Vận hành BGE-Reranker-v2 thông qua thư viện Sentence-Transformers. (Yêu cầu ~ 4GB).
- **Cơ sở Dữ liệu cục bộ:**
 - SQLite FTS5 (In-memory/SSD)
 - LanceDB (In-memory/SSD)
 - Các DB hoạt động bên ngoài VRAM nhằm tránh gây hiện tượng OOM (Out Of Memory).



Thank you!

Hệ thống T-RAG

